

Assignment 7: SyncTask — Kanban Task Board Pending



Objective

Build a **FastAPI-based Kanban Task Board API** to manage projects and tasks. Tasks should move through different workflow stages such as Todo, Doing, and Done.

The assignment should demonstrate:

- Pydantic Field Validation
- SQLAlchemy Relationship Mapping
- One-to-Many Relationships
- Enum Validation
- Cascade Delete
- Path Parameters
- Task Status Management

Technical Requirements

- **Framework:** FastAPI
 - **ORM:** SQLAlchemy
 - Relationship: **One Project → Many Tasks**
 - Task Status Enum:
 - todo
 - doing
 - done
 - Priority range: 1–5
 - Deleting a project must automatically delete all its associated tasks.
-

Data Models

Project

- id
- name
- description

Task

- id
 - title
 - description
 - status
 - priority
 - project_id
-

API Routes

POST /projects

Create a new project workspace.

Required fields:

- Name
 - Description
-

POST /projects/{id}/tasks

Add a new task to a specific project.

Required fields:

- Title
- Description

- Status
- Priority

Validation:

- title minimum length: **3 characters**
 - description maximum length: **500 characters**
 - priority must be between **1 and 5**
 - Status must be a valid Enum value
 - Return 404 Not Found if the project does not exist
-

GET /projects/{id}/tasks

Return all tasks belonging to the selected project.

Tasks should be grouped by status:

todo

doing

done

Example response structure:

```
{  
  "todo": [],  
  "doing": [],  
  "done": []  
}
```

PATCH /tasks/{task_id}

Update the status of an existing task.

Allowed status values:

- todo
- doing
- done

Return 404 Not Found if the task does not exist.

Bonus

Prevent invalid status transitions.

Example:

todo → doing → done

Students may prevent transitions such as:

done → todo

DELETE /projects/{id}

Delete a project.

All tasks associated with that project must also be deleted automatically using **SQLAlchemy Cascade Delete**.

Return 404 Not Found if the project does not exist.

SQLAlchemy Relationship

Implement a **one-to-many relationship** between Project and Task.

Project

↓

Multiple Tasks

Deleting the parent project should automatically remove its associated task records.

Pydantic Validation

Use `Field()` for validation.

Example:

```
title: str = Field(min_length=3)
description: str = Field(max_length=500)
priority: int = Field(ge=1, le=5)
```

Validation & Business Rules

- Task title must contain at least **3 characters**
 - Task description must not exceed **500 characters**
 - Priority must be between **1 and 5**
 - Task status must be a valid Enum
 - A task must belong to an existing project
 - Deleting a project must delete all associated tasks
 - Invalid status transitions may be prevented as a **bonus**
-

Error Handling

Handle appropriate errors for:

- Project not found
- Task not found
- Invalid status
- Invalid priority
- Invalid title length
- Description exceeding maximum length
- Invalid task status transition

Use appropriate HTTP status codes such as 400, 404, and 422.

API Testing

All APIs should be testable through FastAPI Swagger:

/docs

Test:

- Project creation
- Task creation

- Field validation
 - Priority validation
 - Task grouping by status
 - Task status updates
 - Project deletion
 - Cascade deletion of tasks
 - Invalid status transitions, if implemented
-

Submission Instructions

Submit the **complete project as ONE ZIP file through the LMS.**

The ZIP file should include:

- Complete source code
- `requirements.txt`
- `README.md`
- All files required to run the project

ZIP File Name

`StudentName_SyncTask.zip`

Example:

`RahulKumar_SyncTask.zip`

Important

- Submit only **one ZIP file**.
- Do not upload individual files separately.
- Do not include `venv`, `.venv`, `__pycache__`, or unnecessary folders.
- Test the application before submission.
- Ensure the correct ZIP file is uploaded successfully to the LMS.

- Submit before the deadline mentioned in the LMS.
-

Evaluation Criteria

The assignment will be evaluated based on:

- Correct API functionality
- Pydantic Field validation
- SQLAlchemy relationship mapping
- One-to-many relationship implementation
- Task status Enum
- Priority validation
- Task grouping by status
- Cascade delete implementation
- Error handling
- Code quality and organization
- Successful project execution
- Proper ZIP submission

Expected Outcome

By completing this assignment, you should be able to build a structured **FastAPI Kanban Task Board using SQLAlchemy relationships, Pydantic validation, Enum-based task status, cascade delete, and task workflow management.**

Upload Assignment